

RAG DOCUMENT QA WITH CITATION VERIFICATION

A Retrieval-Augmented Generation Pipeline That Verifies Its Own Answers

Project 5 of the AI Engineering Portfolio | Build Guide & Interview Companion
Author: Anas Amiar

"Most RAG demos stop at retrieve-and-generate. This one adds the layer that actually matters in production: checking whether the answer is grounded in what was retrieved, and whether what was retrieved even addressed the question."

How to Use This Guide

This guide documents the RAG Document QA pipeline exactly as it was built: a short pitch, the tech stack, a phase-by-phase build order with reasoning for why each phase comes before the next, and the talking points worth memorizing before an interview.

Read top to bottom to learn the project from scratch, or jump to the "Interview Talking Point" box at the end for a quick pre-call refresher.

What You're Building

A RAG pipeline over a 6-document knowledge base (a fictional SaaS product: pricing, API docs, webhooks, integrations, support SLA, data policy). For every question, the pipeline retrieves the most relevant chunks, generates an answer from them, then runs a two-axis grounding check: did the answer's key terms actually come from the cited chunks, and did the cited chunks actually contain what the question was asking about? Fail either axis — answer is flagged as low-confidence. Every failure is logged with its failure type so you can tell whether to add more documents or fix the generator.

Demo result on 14 questions (10 in KB, 4 outside): **9 answered confidently (64%)**, 3 low-confidence (including a GraphQL false-positive caught by question coverage), 2 cleanly refused.

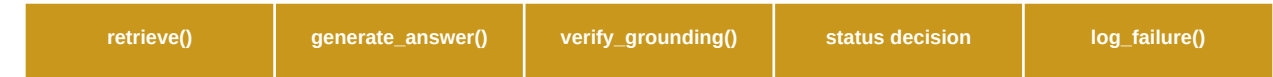
WHY THIS PROJECT LANDS INTERVIEWS

RAG is one of the most common AI system patterns right now, and almost every candidate who builds a RAG demo stops at retrieve-and-generate. This project goes one step further — the step that actually matters for production: verifying that the answer is grounded. The two-axis check (answer grounding + question coverage) demonstrates that you understand both failure modes of RAG systems, not just how to wire them together. And the failure analytics that distinguish retrieval gaps from grounding failures shows you can design observability into an AI system, not just build the happy path.

Tech Stack

Layer	Tool / Library	Purpose
Language	Python 3.11+	Core implementation
Data validation	Pydantic v2	Typed contracts: Chunk, RetrievedChunk, CitationCheck, QaResponse
Embedder (mock)	Bag-of-words TF vectors	Keyword-overlap baseline — zero API cost, zero latency
Embedder (real)	Any embedding API	Swap embed() in embedder.py — ~5 lines
Vector store	In-memory dict	Index at startup; replace with Chroma/pgvector for persistence
Generator (mock)	Extractive QA (sentence selection)	Answer is a real substring of retrieved chunks
Generator (real)	Any LLM (GPT-4o, Claude)	Stub in generator.py, ~10 lines to wire
Grounding check	TF cosine on answer + question terms	Two-axis: answer grounding + question coverage
Failure logging	JSON append-only	Separates retrieval gaps from grounding failures

The Flow for One Question



Top-k chunks by cosine similarity → extractive answer from best-matching sentence(s) → two-axis grounding check (min score = answered / low_confidence / cannot_answer) → log failures by type.

Step-by-Step Build Guide

Six phases, in the order they were actually built. The knowledge base and chunker had to come first because everything else depends on real chunks with real content. The embedder had to work before the retriever could score anything. The generator had to be extractive so the verifier had genuine signal — a template answer would produce artificially perfect grounding scores.

PHASE 1: Knowledge Base + Chunker

Day 1

Goal: Build the document corpus and split it into retrievable chunks before writing any retrieval logic.

- Write 6 knowledge-base documents in ``data/documents.py``: pricing tiers, API auth, data retention, webhooks, Slack/Jira integrations, and support SLA. Varied enough to produce real retrieval ambiguity.
- Define typed shapes: ``Document`` (doc_id, title, text, source) and ``Chunk`` (chunk_id, doc_id, doc_title, text, start_char, end_char).
- Chunker: split at sentence boundaries with character-level overlap (chunk_size=200, overlap=50) so a sentence that straddles two chunks is still fully represented.

Deliverable: A chunked, typed knowledge base ready to be indexed.

PHASE 2: Embedder + Vector Store

Day 1

Goal: Build the retrieval layer — index chunks, score queries.

- Bag-of-words TF vectors with stop-word filtering (question words, domain-universal terms like 'acme' and 'platform' removed — they appear everywhere and carry no signal).
- Cosine similarity between query vector and chunk vectors.
- ``VectorStore``: add chunks at startup, retrieve top-k at query time.
- Expose ``build_store(documents)`` as the single indexing entry point.

Deliverable: ``rag/embedder.py`` + ``rag/retriever.py`` — ``store.retrieve(question, top_k=3)`` returning ranked ``RetrievedChunk`` objects with similarity scores.

Goal: Produce answers that are genuine substrings of retrieved chunks — not fake templates.

- Find the sentence in the top retrieved chunk with the highest overlap with the question.
- Return that sentence plus the next one for context — two sentences max.
- If top-chunk similarity is below threshold (0.05), return a clean 'cannot find relevant information' refusal instead of guessing.
- Stub a real-mode path: replace the extractive logic with an LLM chat completion call.

Deliverable: ``rag/generator.py`` — extractive answers where every term is traceable to a cited chunk.

PHASE 4: The Citation Verifier

Day 2

Goal: The core of the project — check whether the answer is actually grounded in what was retrieved.

- **Axis 1 — Answer grounding:** what fraction of the answer's content terms appear in the cited chunks? Catches hallucinated facts the chunks never mentioned.
- **Axis 2 — Question coverage:** what fraction of the QUESTION's content terms appear in the cited chunks? Catches retrievals that returned something plausible but wrong.
- Final score = $\min(\text{answer_grounding}, \text{question_coverage})$. Fail either axis, fail the check.
- Grounding threshold: 0.60. Below it, the answer is low_confidence.

Deliverable: ``rag/verifier.py`` — ``verify_grounding(answer, cited_chunks, question)`` returning a ``CitationCheck`` with `is_grounding`, `score`, and which terms were ungrounded.

PHASE 5: The Pipeline + Feedback Loop

Day 2-3

Goal: Wire everything into one entry point and log every failure with its failure type.

- ``ask(question, store)`` chains `retrieve` → `generate` → `verify` → status decision (`answered` / `low_confidence` / `cannot_answer`).
- Every `low_confidence` or `cannot_answer` result is logged to ``reports/eval_dataset.json`` with its failure type: ``retrieval_gap`` (the KB doesn't cover this topic) or ``grounding_failure`` (the KB has it, but retrieval brought back the wrong chunk).
- ``failure_analytics()`` shows which type dominates — that tells you WHERE to invest: add documents (retrieval gap) or improve the generator/embedder (grounding failure).

Deliverable: ``rag/pipeline.py`` + ``rag/feedback.py`` — the full end-to-end path plus a failure log that distinguishes knowledge gaps from system gaps.

PHASE 6: The Batch Dashboard

Day 3

Goal: Run a realistic question set and report the numbers a team would actually care about.

- 14 questions: 10 answerable from the KB, 4 outside it.
- Report: answer rate %, avg grounding score on answered questions, failure breakdown.
- Verify the system catches the GraphQL false-positive (retrieval brings back the Support SLA doc; question coverage correctly flags 'graphql' as uncovered).

Deliverable: ``rag/dashboard.py`` — batch run with aggregate stats proving the grounding check adds real value, not just overhead.

Demo Results

Status	Count	What it means
answered	9 (64%)	In-KB questions answered and grounded (score >= 0.60)
low_confidence	3	GraphQL question, mobile app, rate-limit edge case
cannot_answer	2	"Capital of France" + "What language is Acme built in?"
Failure breakdown	3 grounding / 2 retrieval	Tells you which part of the system to fix
Avg grounding score	0.81	On the 9 answered questions — high citation quality

Architecture Decisions

Why two grounding axes instead of one?

Answer grounding alone misses the case where retrieval succeeds (a real chunk is cited) but returned the wrong thing. A question about 'GraphQL support' might retrieve a chunk from the Support SLA document — the answer is 'grounded' in that chunk, but it doesn't address the question. Question coverage catches this: the question's key term 'graphql' isn't in any cited chunk, so the coverage score is low and the answer gets flagged.

Why extractive QA in mock mode?

An extractive answer is a genuine substring of the retrieved chunks. This means the grounding check has real signal: if the answer were a template, all its terms would trivially be 'grounded' and the check would produce artificially perfect scores. Extractive answers produce genuine variation in grounding scores because sentence selection isn't perfect.

Why log retrieval gap vs. grounding failure separately?

These require different fixes. A retrieval gap means the knowledge base doesn't cover the topic — fix by adding documents. A grounding failure means the KB has the answer but the system didn't use it correctly — fix by improving the embedder, the chunking strategy, or the generator prompt. Conflating them produces analytics that point at the wrong fix.

Deliberately Out of Scope for v1

- Real embedding API (swap ``embed()`` in ``embedder.py`` — one function, ~5 lines).
- Real LLM generation (swap the mock branch in ``generator.py`` — ~10 lines).
- Persistent vector store (currently in-memory — replace ``VectorStore`` with Chroma or pgvector).
- Re-ranking: retrieve top-10, re-rank to top-3 with a cross-encoder.
- A UI for browsing and labeling the failure eval dataset.

INTERVIEW TALKING POINT

"What was the hardest design decision?"

Adding the question coverage axis. The natural first version just checks answer grounding — are the answer's terms in the cited chunks? That's useful, but it's blind to the most common real-world RAG failure: retrieval brings back a plausible-looking chunk that doesn't actually address the question. The system answers confidently, the answer is 'grounded', and it's still wrong. Adding question coverage — do the cited chunks even contain what the question asked about? — is what makes the grounding check actually catch that failure mode. It's the difference between checking 'did the LLM stick to the script' and checking 'was the right script retrieved in the first place.'

What I'd Build Next

- Swap ``embed()`` for a real embedding API (OpenAI, Cohere) for semantic retrieval — the 'support' false-positive would vanish with real embeddings.
- Wire in a real LLM for generation with a system prompt constraining it to only use retrieved context.
- Re-ranking: retrieve top-10, re-rank to top-3 with a cross-encoder, then generate.
- A persistent vector store (Chroma, pgvector) so the index survives restarts.

Companion Projects in This Portfolio

This project is the 'can the system answer reliably?' layer — which pairs with the **Failure Forensics Tool** (Project 3), which asks 'when a pipeline step fails, which step was it?' Both build AI systems that inspect and explain their own failures instead of silently returning bad output. Together with the **Model Regression Detection System** (Project 1), the **LLM Cost Autopilot** (Project 2), and the **OCR Document Intelligence Pipeline** (Project 4), these five projects cover correctness, cost, observability, uncertainty-aware automation, and retrieval reliability — the full practical stack of concerns for shipping AI in production.

Repository: github.com/Anas-Amiar/Project-5-rag-document-qa